

Platform Intelligence Engine (PIE)

Operations Runbook

Package: adpie (v0.3.0) | Generated August 27, 2026

PIE - Runbook

- Platform Intelligence Engine (PIE) — Operations Runbook
 - 1. Prerequisites
 - 2. First-Time Setup
 - 2.1 One-shot bootstrap (recommended)
 - 2.2 Manual setup (if you need to do it by hand)
 - 2.3 Environment configuration
 - 3. Starting the Application
 - 3.1 Start everything together (recommended for local dev)
 - 3.2 Start components individually
 - 3.3 Verify it's up
 - 4. Standard Operating Flow (Happy Path)
 - 4.1 CLI-only flow (no frontend)
 - 5. Running Tests
 - 6. Routine Maintenance Tasks
 - 7. Troubleshooting Guide
 - 7.1 Backend won't start / import errors
 - 7.2 Frontend shows login loop / doesn't remember factory selection
 - 7.3 Chat / AI errors: "Sorry, I encountered an error. Please ensure the backend is running and synced"
 - 7.4 Azure authentication failures
 - 7.5 Graph / dependency-analysis returns unexpected or empty results
 - 7.6 High latency / slow responses
 - 8. Deployment Considerations (Beyond Local Dev)
 - 9. Escalation / Where to Look Next

Platform Intelligence Engine (PIE) — Operations Runbook

Companion to `TECHNICAL_DOCUMENTATION.md`. This runbook covers setup, day-to-day operation, and troubleshooting for the `adpie` / `pie` project (`PlatformIntelligenceEngine-main`).

1. Prerequisites

Requirement	Version	Notes
Python	≥ 3.10 (3.11 recommended)	Backend runtime; project uses a <code>src/</code> layout package.
Node.js	≥ 18	Frontend (Vite/React) build & dev server.
npm	bundled with Node	Frontend package manager.
pip or uv	latest	Python dependency installer.
Azure account	Reader role minimum	Needed to discover real Data Factories (optional — <code>mock</code> auth mode works offline).

Ports used locally: - **8000** — FastAPI backend (`uvicorn`) - **8100** — Persistent OAuth2 PKCE callback server (started automatically inside the backend process) - **5173** — Vite frontend dev server

2. First-Time Setup

2.1 One-shot bootstrap (recommended)

```
# Linux / macOS / Git Bash
./setup.sh
```

```
# Windows PowerShell
.\setup.ps1
```

This performs, in order: 1. Creates `.venv` if missing. 2. Upgrades `pip`. 3. `pip install -r requirements.txt`. 4. `pip install -e .` — installs the `pie` package in editable mode (**required** because of the `src/` layout; without this, `import pie` fails everywhere). 5. `npm install` inside `frontend/` (if `frontend/package.json` exists).

Safe to re-run at any time (idempotent).

2.2 Manual setup (if you need to do it by hand)

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -e .              # registers `pie` as an importable package
cd frontend && npm install && cd ..
```

2.3 Environment configuration

```
cp .env.example .env
```

Edit `.env` and set at minimum `PIE_AUTH_MODE`. For a fully offline/demo run, set:

```
PIE_AUTH_MODE=mock
```

For a real Azure connection, leave `PIE_AUTH_MODE=default` (uses `DefaultAzureCredential`, which tries environment vars → managed identity → Azure CLI login → interactive, in order) or set `PIE_AUTH_MODE=interactive / device_code / cli / service_principal` explicitly. See the full variable table in the Technical Documentation §10.

If you intend to use AI chat features, either: - populate `AZURE_AI_FOUNDRY_*` / provider keys in `.env`, **or** - leave them blank and add keys later from the web UI (Sidebar → Settings → API Keys), which calls `POST /api/v1/settings/keys`, **or** - do nothing — the system automatically falls back to `DeterministicMockLLMProvider` (offline, deterministic, no cost) if no provider key resolves. You'll see a log line: `Error creating LLM provider ... Falling back to Mock Provider`. This is expected and non-fatal.

3. Starting the Application

3.1 Start everything together (recommended for local dev)

```
./start.sh      # Linux/macOS/Git Bash
```

```
.\start.ps1     # Windows PowerShell
```

Starts backend and frontend concurrently, prints both URLs, and stops both on `Ctrl+C`. Requires `setup.sh / setup.ps1` to have run at least once (checks for `.venv` and `frontend/node_modules`).

Override default ports with environment variables before running:

```
BACKEND_PORT=8001 FRONTEND_PORT=5174 ./start.sh
```

3.2 Start components individually

Backend only:

```
# with venv active
uvicorn pie.api.app:app --reload --host 0.0.0.0 --port 8000

# or without activating, call the venv python directly
.venv/bin/python -m uvicorn pie.api.app:app --reload --host 0.0.0.0 --port 8000
```

Or via the CLI:

```
pie serve --host 0.0.0.0 --port 8000 --reload
```

Frontend only:

```
cd frontend
npm run dev
```

3.3 Verify it's up

```
curl http://localhost:8000/health
# {"status":"HEALTHY","service":"PIE-Core-Platform","version":"3.0.0"}
```

- Backend Swagger UI: <http://localhost:8000/docs>
- Frontend: <http://localhost:5173>

4. Standard Operating Flow (Happy Path)

1. Open the frontend at <http://localhost:5173>.
2. **Login** — the Setup Wizard sends you through Microsoft OAuth2 PKCE login (backend `POST /auth/login` → browser redirect → local callback server on `:8100` → `GET /auth/poll/{flow_id}`).
3. **Select subscription(s)** — wizard calls `GET /api/v1/subscriptions`.
4. **Select factory/factories** — wizard calls `POST /api/v1/subscriptions/factories`, then `POST /api/v1/discovery/sync` to pull and cache metadata. This step always re-runs after every login by design (ensures fresh sync — see §7.2 below), even if factories were previously synced.
5. **Explore** — Asset Explorer (`FactoryOverview` / `DataCanvas`) browses pipelines/datasets/linked services/triggers via the discovery endpoints, and the dependency graph via `/graph/*` endpoints.
6. **Ask AI questions** — Chat drawer streams from `POST /api/v1/ai/chat/stream` (or single-shot `/ai/ask`), grounded in the synced factory's knowledge graph.
7. **Run impact/deletion analysis** — Change Impact panel calls `POST /api/v1/graph/change-impact` or `POST /api/v1/graph/deletion-simulation`.
8. **Review audits** — Technical debt, concurrency heatmap, SaaS vendor map via `/api/v1/audit/*`.

4.1 CLI-only flow (no frontend)

```
pie discover          # interactive subscription/factory discovery + sync
pie audit             # technical debt / vendor / concurrency summary
pie simulate-deletion <asset_name>
pie chat              # terminal AI assistant
pie clear             # wipe in-memory workspace
```

5. Running Tests

```
# full suite
.venv/bin/python -m pytest tests/ -v

# just API endpoint tests
.venv/bin/python -m pytest tests/api/ -v

# just unit tests
.venv/bin/python -m pytest tests/unit/ -v

# a single file
.venv/bin/python -m pytest tests/unit/test_graph_engine.py -v
```

Tests do not require live Azure credentials — auth/AI dependencies are mocked/fixtures-driven (`tests/api/conftest.py` , `tests/conftest.py` , `pie.ai.providers.DeterministicMockLLMProvider`).

Ad hoc top-level smoke scripts (run directly, not via pytest):

```
.venv/bin/python test_factory_workflow.py
.venv/bin/python test_pipeline_queries.py
.venv/bin/python test_workflow_detailed.py
.venv/bin/python check_repo.py
```

6. Routine Maintenance Tasks

Task	Command / Location
Re-sync a factory's metadata	UI: Sidebar → factory refresh, or <code>POST /api/v1/factories/{name}/refresh</code> , or <code>pie discover</code>
Clear all cached/in-memory factories	<code>pie clear</code> , or delete the on-disk cache directory referenced by <code>PIE_OUTPUT_DIR</code>
Rotate/update LLM provider API keys	UI Settings modal → <code>POST /api/v1/settings/keys</code> , or edit <code>.env</code> and restart backend
Update backend dependencies	Edit <code>requirements.txt</code> / <code>pyproject.toml</code> , then <code>pip install -r requirements.txt && pip install -e .</code>
Update frontend dependencies	Edit <code>frontend/package.json</code> , then <code>cd frontend && npm install</code>
Regenerate <code>adpie.egg-info</code> (editable install metadata)	<code>pip install -e .</code> (auto-regenerates)
View live logs	Backend terminal (Rich-formatted); adjust verbosity with <code>PIE_LOG_LEVEL</code>

7. Troubleshooting Guide

7.1 Backend won't start / import errors

Symptom	Cause	Fix
<code>ModuleNotFoundError: No module named 'pie'</code>	Package not installed in editable mode	<code>pip install -e .</code> from repo root (with venv active)
<code>uvicorn: The term 'uvicorn' is not recognized</code> (Windows)	Venv not active, or installed to global Python	Activate venv (<code>.venv\Scripts\Activate.ps1</code>) or call <code>.venv\Scripts\python.exe -m uvicorn ...</code> directly
Error creating LLM provider ... No module named 'openai'	<code>openai</code> SDK missing from venv	<code>pip install -r requirements.txt</code> (it's already listed — reinstall if venv is stale)
Backend starts but <code>/health</code> doesn't respond	Wrong port, or another process bound to 8000	Check <code>netstat /lsof -i :8000</code> ; use <code>BACKEND_PORT</code> override

7.2 Frontend shows login loop / doesn't remember factory selection

This was previously an actual bug, fixed and documented in `FIXES_IMPLEMENTED.md`. Current **intended** behavior: after **every** login, the app always re-shows subscription → factory selection (this is deliberate — it guarantees fresh metadata and lets users switch factories). If you expected it to skip straight to the workspace and it doesn't, that's correct/by-design, not a bug.

If factory selection appears to not “stick” for the AI chat feature specifically, verify `localStorage` has a `selected_factory` key (DevTools → Application → Local Storage) — `ChatContainer.jsx` reads this rather than being hardcoded (previously it was hardcoded to `'adf-sales-enterprise-prod'`; this was fixed).

7.3 Chat / AI errors: “Sorry, I encountered an error. Please ensure the backend is running and synced”

This is the primary documented failure mode (see `CHAT_ERROR_DIAGNOSTICS.md`). Diagnose in this order:

Step 1 — Reload the frontend (`Ctrl+R` / `F5`).

Step 2 — Open browser DevTools console (`F12`) and reproduce: log in → select subscription/factory → ask a question (e.g. “How many pipelines?”).

Step 3 — Read the console output. You'll see one of three message shapes:

- *Request details logged* (factory, model, `sessionToken: "✓ present"`, query) — confirms the request left the browser correctly; if it still fails, the problem is server-side or network.
- *Successful response* (`detected_intent`, `response_markdown`, `latency_ms`) — the call actually succeeded; if the user still saw an error, it was likely transient/intermittent.
- *Error object* (`message`, `status`, `statusText`, `data.detail`) — **this is the one that matters**; `data.detail` contains the real backend error message.

Step 4 — Match the error to a cause:

Error text / symptom	Likely cause	Fix
Factory not found	Selected factory was never synced	Settings → re-sync factories, or run through onboarding again
Graph is empty	Sync ran but no metadata loaded (e.g., empty ADF or permission gap)	Confirm the factory actually has pipelines; check Azure Reader RBAC on that resource group
Missing API key	Selected non-mock LLM model (Google Gemini, NVIDIA NIM, Azure OpenAI) needs a key not yet configured	Settings → add the relevant provider API key, or switch model to a configured/mock provider
Network Error / connection refused	Backend not running or crashed	Check backend terminal; restart with <code>uvicorn pie.api.app:app --reload</code>
CORS error in console	Frontend/backend port mismatch (not using default <code>5173</code> / <code>8000</code>)	Confirm both are on expected ports, or that <code>CORSMiddleware</code> origins weren't restricted

Step 5 — Test the backend directly, bypassing the frontend, to isolate whether the issue is server-side or client-side:

```
import requests
url = "http://localhost:8000/api/v1/ai/ask"
payload = {
    "query": "How many pipelines do we have?",
    "factory_name": "<your-synced-factory-name>",
    "model": "azure-openai", # or whichever provider you've configured
}
r = requests.post(url, json=payload)
print(f"Status: {r.status_code}")
print(f"Response: {r.json()}")
```

If this returns `200`, the backend and reasoning pipeline are healthy — the issue is in the frontend layer (stale session token, wrong factory name in `localStorage`, or a CORS mismatch).

Pre-flight checklist before escalating a chat error: - [] Backend terminal shows `Application startup complete` with

no errors - [] Frontend loaded (PIE logo visible) - [] Login succeeded (subscriptions listed) - [] A factory was selected and sync completed (no spinner stuck on “Syncing...”) - [] Workspace/chat interface is visible - [] `localStorage` has a `selected_factory` key

7.4 Azure authentication failures

Symptom	Cause	Fix
Login popup fails / redirect doesn't complete	Port 8100 blocked or already in use by another process	Free port 8100, or check firewall rules for localhost callback
<code>PieAuthError</code> / <code>PiePermissionError</code> from <code>rbac_discovery.py</code>	Insufficient RBAC (needs at least Reader) on target subscription	Request Reader role from Azure admin
Want to test without any Azure tenant	—	Set <code>PIE_AUTH_MODE=mock</code> in <code>.env</code> ; uses <code>MockTokenCredential</code> for a fully offline demo
Device-code flow needed (headless/server)	Interactive browser not available	Set <code>PIE_AUTH_MODE=device_code</code>

7.5 Graph / dependency-analysis returns unexpected or empty results

- Confirm the factory was actually synced (`GET /api/v1/factories` should list it) before calling any `/graph/*` or `/audit/*` endpoint — these operate on the in-memory graph built from synced metadata, not live Azure calls.
- If an asset name lookup fails, note that `AssetQueryEngine` / `QueryIntentRouter` use fuzzy matching (`difflib`) — verify the exact asset name via `GET /api/v1/pipelines` or `/datasets` first.
- After a factory refresh, downstream services (graph, context, AI engine) are rebuilt lazily when a different factory is requested (`pie.ai.engine`); if you refreshed metadata but a subsequent question still reflects stale data, restart the backend to force a full rebuild.

7.6 High latency / slow responses

- Real LLM providers (OpenAI/Azure OpenAI/Anthropic/Gemini/NVIDIA) will have network + inference latency; the `DeterministicMockLLMProvider` responds in <300ms by design — if you need very fast automated testing/demos, explicitly select the mock provider.
- For large factories, initial `discovery/sync` can take longer due to ADF API pagination — this is expected; subsequent reads are served from the fast in-memory repository.

8. Deployment Considerations (Beyond Local Dev)

The shipped configuration is optimized for **local development / demo**, not production. Before deploying beyond a single developer machine, address:

1. **CORS** — `allow_origins=["*"]` in `pie/api/app.py` must be restricted to known frontend origin(s).
2. **Session store** — `pie.auth.session_store` is in-process memory; it will not survive restarts or scale across multiple backend instances. Migrate to Redis/Azure Cache for HA (flagged in the codebase’s own docstring as a planned Phase 5+ item).
3. **Secrets** — never commit `.env` / `src/.env`; use a secret manager (Azure Key Vault, etc.) in real deployments.
4. **OAuth redirect URI** — the callback server is hardcoded to `http://localhost:8100`, tied to the public Azure CLI client ID. A production deployment needs its own registered Entra ID app + redirect URI strategy rather than reusing the Azure CLI public client.
5. **Process management** — `start.sh` / `start.ps1` are dev convenience scripts (`uvicorn --reload + vite dev`); use a production ASGI setup (e.g., `uvicorn/gunicorn` with workers, behind a reverse proxy) and a built (`npm run build`) static frontend instead.
6. **Least privilege in production Azure** — keep the service principal / managed identity scoped to Reader only, per the project’s core security principle.

9. Escalation / Where to Look Next

If you need...	Look here
Full module-by-module architecture	TECHNICAL_DOCUMENTATION.md (this repo's companion doc) or PIE_MASTER_CONTEXT.md
Phase-by-phase product/requirements history	Documentation/Phases/01-07_*.md , Documentation/Requirement/*.docx
Prior deep-dive source analysis	CODEBASE_ANALYSIS.md
History of specific bug fixes already applied	FIXES_IMPLEMENTED.md
Original chat-error debugging notes	CHAT_ERROR_DIAGNOSTICS.md (source for §7.3 above)
Live API contract	http://localhost:8000/docs (Swagger) while the backend is running